TACHYS

Trading Architecture for Crypto & High-Yield Systems

Low-Latency Market-Making, Arbitrage & Prediction Market System

Complete Technical Specification & Learning Roadmap

Version 1.0 • March 2026

Target Duration: 12–18 Months

CONFIDENTIAL

Table of Contents

1. Executive Summary & Strategic Rationale
2. System Architecture Overview
3. Phase 0: Environment, Tooling & C++ Re-Immersion (Weeks 1–2)
4. Phase 1: Core Feed Handler — Single-Threaded Foundation (Weeks 3–8)
5. Phase 2: Concurrent Pipeline — Lock-Free Multithreading (Weeks 9–16)
6. Phase 3: Order Book Engine & Strategy Logic (Weeks 17–24)
7. Phase 4: Execution Engine & Order Management System (Weeks 25–34)
8. Phase 5: Risk Management & Production Hardening (Weeks 35–42)
9. Phase 6: Multi-Venue Expansion & Polymarket Integration (Weeks 43–52)
10. Phase 7: AI/ML Signal Layer & Agent Orchestration (Weeks 53–64)
11. Phase 8: Replay, Backtesting & Performance Lab (Weeks 65–72)
12. C++ Refresher Reference — Key Concepts Explained
13. AWS Deployment Architecture
14. Revenue Model & Realistic Expectations
15. Appendices: Reading List, API References, Interview Preparation

1. Executive Summary & Strategic Rationale

TACHYS (Trading Architecture for Crypto & High-Yield Systems) is a production-grade, low-latency trading system built entirely in modern C++ (C++20/23). It connects to cryptocurrency exchanges and prediction markets (Polymarket), performs real-time market-making, cross-venue arbitrage, and AI-driven directional trading. It is designed to be both a rigorous educational vehicle for mastering systems-level C++ and a genuine revenue-generating platform.

This specification is written for a developer returning to C++ after working primarily in Python. Every component includes not just what to build, but **why** each design choice matters in a low-latency context, and brief refreshers on the C++ features involved. The goal is that you can hand any section of this document to an AI assistant or reference it yourself and have full context to implement it.

1.1 Why This Project

Crypto and prediction markets are the only asset classes where a solo developer can connect to real exchanges, submit real orders, and run a live system with zero institutional gatekeeping. The technical challenges are identical to those solved at Citadel, Jane Street, Two Sigma, and Jump Trading: order book reconstruction, lock-free messaging, sub-microsecond hot paths, custom memory management, and risk-controlled execution. The difference is that you own the entire stack and can point to real P&L.

1.2 What Makes This Different From a Toy Project

Most side projects stop at "read data and detect an opportunity." TACHYS goes through to execution, risk management, multi-venue coordination, and AI-driven signal generation. The system is designed so that the same binary runs in both live and replay mode — a hallmark of professional trading systems. Every phase produces a working, testable deliverable, not just library code.

1.3 Career Impact

In a hedge fund interview, candidates typically demonstrate algorithmic skill through LeetCode-style problems. TACHYS lets you instead walk in with a live system that has generated real P&L. You will have direct experience with every topic interviewers at low-latency firms probe: **cache line optimization**, **lock-free concurrency**, **tail latency measurement**, **order book microstructure**, **memory allocation strategies**, and the **operational reality** of running a system that handles real money. The war stories alone — the race condition that caused a double-fill, the exchange outage mid-position, the p99 spike from a GC in a dependent process — are worth more than any number of whiteboard exercises.

2. System Architecture Overview

TACHYS is a layered system with a strict separation between the **hot path** (latency-critical, allocation-free, single-threaded per venue) and the **warm/cold paths** (risk management, logging, monitoring, AI inference). The layers communicate exclusively through lock-free queues and shared memory — no mutexes ever appear on the critical path.

2.1 Architectural Layers

Hot Path (< 1μs)	WebSocket recv → Parse → Book Update → Strategy Signal → Risk Check → Order Creation. Runs pinned to an isolated CPU core. Zero heap allocations. No virtual dispatch. No system calls except network I/O.
Warm Path (< 100μs)	Order Management System (OMS), position tracking, fill reconciliation, P&L calculation. Runs on a dedicated core. Communicates with the hot path via SPSC lock-free queues.
Cold Path (< 10ms)	Async logging, monitoring/metrics export, state snapshots, configuration reload. Runs on shared cores. Communicates via MPSC queues.
Intelligence Layer	Python-based AI/ML signal generation. Publishes fair-value estimates to C++ via shared memory (mmap). Runs on separate cores or a separate machine. Latency budget: seconds, not microseconds.
Operations Layer	Web dashboard (Python/FastAPI), Grafana monitoring, alerting, deployment scripts. Not latency-sensitive.

2.2 Data Flow

The system follows a strict left-to-right data flow with no back-pressure on the hot path:

1. Exchange WebSocket sends market data update (JSON or binary).

2. I/O thread receives raw bytes, parses into an internal MarketDataEvent struct using zero-copy techniques.

3. MarketDataEvent is published to an SPSC ring buffer (hot path queue).

4. Strategy thread consumes events, updates the internal order book, evaluates strategy signals.

5. If a signal fires, an OrderRequest is created (from the object pool, no allocation) and pushed to the OMS queue.

6. OMS thread manages order lifecycle: submission to exchange, tracking acks/fills/cancels, position updates.

7. All threads publish log events to the async MPSC logging queue.

8. A monitoring thread periodically samples latency histograms and exports metrics.

2.3 Key Design Principles

Zero-Allocation Hot Path

Every object used on the critical path is pre-allocated at startup. The hot path never calls new, malloc, or any allocating container operation (push_back that triggers resize, string concatenation, etc.). All dynamic-sized data is handled via fixed-capacity containers and object pools.

Mechanical Sympathy

Data structures are designed for how CPUs actually work: cache-line-aligned structs (64 bytes on x86), contiguous memory layouts for sequential access, and avoidance of pointer-chasing data structures (linked lists, tree maps) on the hot path. The goal is to keep the working set in L1/L2 cache.

Deterministic Latency Over Throughput

We optimize for p99/p99.9 latency, not average throughput. A system that processes 99% of messages in 500ns but spikes to 50μs on the 99.9th percentile is worse than one that consistently runs at 2μs. This means no locks, no unbounded allocations, no system calls on the hot path.

3. Phase 0: Environment, Tooling & C++ Re-Immersion

Weeks 1–2 • Goal: Set up a professional C++ development environment and shake off the rust.

3.1 Development Environment

Component	Objective	C++ Low-Level Focus	Why It Matters
Compiler	Install GCC 13+ or Clang 17+ with full C++20/23 support.	Ensure support for concepts, std::expected, constexpr containers, and modules (experimental). Verify with a small test program using std::atomic, std::jthread, and std::span.	Modern C++ features eliminate entire categories of bugs and enable zero-cost abstractions that older C++ could not express.
Build System	Configure CMake 3.28+ with presets for Debug, Release, and Benchmark builds.	Release flags: -O3 -march=native -flto -DNDEBUG. Debug flags: -O0 -g -fsanitize=address,undefined . Add a Benchmark preset with -O3 -g for profiling with symbols.	Correct compiler flags can yield 2–5x performance differences. -march=native enables CPU-specific SIMD. LTO enables cross-translation-unit inlining.
Dependencies	Install Boost 1.84+ (for Asio), Google Benchmark, Google Test, fmt library, and simdjson.	Use CMake FetchContent or vcpkg for dependency management. Pin exact versions for reproducibility.	simdjson parses JSON at gigabytes/second using SIMD — it will be your primary parser. fmt is allocation-free formatted output.
Profiling Tools	Install and verify perf, Valgrind (cachegrind/callgrind), and optionally Intel VTune.	Run perf stat on a simple loop to verify PMU access. Run cachegrind on a matrix multiply to see cache miss reporting.	You will use perf extensively in Phase 4+. Getting comfortable with it now saves frustration later.
IDE / Editor	Configure your editor with clangd for C++20-aware autocompletion and diagnostics.	Ensure clangd picks up the compile_commands.json generated by CMake. Configure format-on-save with a .clang-format file.	clangd catches errors that the compiler misses until link time. A consistent code style prevents merge conflicts with yourself.

3.2 C++ Warm-Up Exercises

Before touching the real project, spend 3–4 days on these exercises to rebuild muscle memory. Each targets a specific skill used heavily in later phases.

> **Exercise 1: RAII Socket Wrapper**
>
> Write a class that opens a TCP socket in the constructor and closes it in the destructor. Implement move semantics (move constructor, move assignment) and delete copy operations. Connect to a public echo server and send/receive a message. This refreshes RAII, move semantics, and Linux socket API basics.

Exercise 2: Lock-Free SPSC Queue

Implement a fixed-capacity single-producer single-consumer ring buffer using std::atomic<size_t> for head and tail indices. Use std::memory_order_acquire/release for the consumer/producer respectively. Write a Google Benchmark that measures throughput (messages/second) and latency (ns/message) with the producer and consumer on separate threads pinned to separate cores. Target: < 50ns round-trip on modern hardware.

Exercise 3: Object Pool Allocator

Create a fixed-size pool that pre-allocates N objects of type T at construction. Implement allocate() and deallocate() methods that operate in O(1) using a free-list (intrusive singly-linked list embedded in the objects themselves). Ensure zero heap allocation after construction. Benchmark against std::allocator.

Exercise 4: simdjson Parsing

Download a sample Binance WebSocket message (order book snapshot). Parse it with simdjson using the On Demand API. Extract bids and asks into a pre-allocated std::array. Benchmark parse time. You should see < 1μs for a typical message.

3.3 Deliverables

At the end of Phase 0 you should have:

1. A CMake project that compiles with C++20, runs Google Test and Google Benchmark.
2. Four passing warm-up exercises with benchmarks showing measured performance.
3. A working perf setup that can profile a running binary.
4. Comfort with std::atomic, std::span, move semantics, RAII, and templates.

4. Phase 1: Core Feed Handler — Single-Threaded Foundation

Weeks 3–8 • Goal: Connect to a live exchange WebSocket, parse market data at wire speed, and reconstruct an L2 order book in a single thread.

4.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
WebSocket Client	Establish a persistent WebSocket connection to Binance or Bybit and receive raw market data frames.	Use Boost.Beast (built on Boost.Asio) for the WebSocket client. Configure TCP_NODELAY to disable Nagle's algorithm. Use a pre-allocated read buffer (64KB) to avoid per-message allocation. Handle ping/pong heartbeats to keep the connection alive.	Nagle's algorithm batches small TCP writes for efficiency, but adds latency. Disabling it is standard in low-latency systems. Pre-allocated buffers prevent the allocator from appearing on the hot path.
Message Parser	Deserialize raw JSON market data into internal C++ structs at the lowest possible cost.	Use simdjson On Demand API for zero-copy JSON traversal. Parse directly into a pre-allocated MarketDataEvent struct. Use std::string_view (never std::string) for symbol names. Use fixed-point integer arithmetic for prices and quantities (multiply by 10^8, store as int64_t) to avoid floating-point comparison issues.	Floating-point equality comparison is unreliable for prices. Fixed-point eliminates this entirely and is faster for arithmetic. simdjson's On Demand API never allocates — it walks the JSON tape in-place.
MarketDataEvent	Define the core data structure that carries parsed market data through the pipeline.	A POD (Plain Old Data) struct, 64 or 128 bytes, cache-line aligned with [[alignas(64)]]. Fields: timestamp (uint64_t nanoseconds since epoch), symbol_id (uint16_t, not a string), side (enum : uint8_t), price (int64_t fixed-point), quantity (int64_t fixed-point), sequence_number (uint64_t), event_type (enum : uint8_t). Add padding bytes explicitly to avoid compiler-inserted padding.	A cache line on x86 is 64 bytes. If your hot struct fits in one or two cache lines, reading it is a single memory fetch. Implicit padding wastes cache space. Explicit padding documents your intent and ensures the layout is what you expect.
Internal Order Book	Maintain the top N levels of the bid and ask sides for one instrument.	Use two std::array<PriceLevel, 20> (fixed-capacity, stack-allocated) for bids and asks. PriceLevel is {int64_t	For small N (< 50 levels), a sorted array beats std::map by 5–10x because the entire book fits in L1 cache. std::map allocates a tree

Component	Objective	C++ Low-Level Focus	Why It Matters
		price; int64_t quantity;}. Updates are applied by scanning the array (linear scan over 20 elements is faster than a tree lookup due to cache locality). Maintain a best_bid / best_ask pointer for O(1) top-of-book access.	node per entry and scatters them across memory, causing cache misses on every lookup.
Sequence Tracking	Detect dropped or out-of-order messages from the exchange.	Track the last sequence number per stream. If a gap is detected, log a warning and request a snapshot resync via REST API. Use std::expected (C++23) or a simple Result<T,E> type to propagate errors without exceptions on the hot path.	Exchanges can and do drop messages, especially during high-volatility periods. A production system must detect this and recover. Exceptions are banned on the hot path because they involve stack unwinding and heap allocation.
Timestamping	Stamp every event with a high-resolution local timestamp for latency measurement.	Read the CPU timestamp counter directly via __rdtsc() intrinsic (x86) or use std::chrono::steady_clock. Calibrate TSC ticks to nanoseconds at startup using a reference measurement. Stamp on entry to the recv() call and on completion of book update to measure wire-to-state latency.	std::chrono::system_clock can have microsecond granularity on some platforms. TSC gives nanosecond resolution with < 10ns overhead per read. However, TSC values are per-core, so you need calibration if threads migrate (which they won't, since we pin them).
Logging (v1)	Log system events (connect, disconnect, errors, periodic stats) without blocking the hot path.	A synchronous, fire-and-forget logger that writes to a pre-allocated circular char buffer. The buffer is flushed to disk by a background timer (every 100ms). Use fmt::format_to() to format directly into the buffer with zero allocation. This is a temporary solution — Phase 4 replaces it with a proper async MPSC logger.	Full async logging is complex. Starting with a simple buffered logger lets you focus on the feed handler. The key constraint is that formatting a log message must not allocate — fmt::format_to() with a fixed buffer guarantees this.

4.2 C++ Concepts Refresher for This Phase

> **std::string_view**
>
> A non-owning reference to a character sequence (pointer + length). It never allocates memory. Use it whenever you need to pass or store a read-only string without copying. Danger: the underlying data must outlive the string_view. In our case, the simdjson parse buffer is alive for the duration of processing, so string_views into it are safe within a single message's processing.

[[alignas(N)]]

Tells the compiler to align the start address of a struct to an N-byte boundary. [[alignas(64)]] ensures the struct starts at a cache line boundary. This prevents false sharing (two threads accessing different structs that happen to share a cache line) and ensures a small struct can be loaded in a single cache fetch.

Fixed-Point Arithmetic

Instead of storing prices as double (e.g., 43521.75), store as int64_t with an implicit scale factor (e.g., 4352175000000 with scale 10^8). All arithmetic is integer arithmetic, which is exact and faster. Comparison is trivial (==, <, > just work). The scale factor is chosen to accommodate the smallest tick size across all target exchanges.

4.3 Acceptance Criteria

1. Connects to Binance WebSocket and receives live BTC/USDT depth stream.
2. Parses messages with simdjson in under 1μs per message (benchmarked).
3. Maintains a correct L2 order book (verified against REST snapshot).
4. Detects and recovers from sequence gaps.
5. Wire-to-book-update latency < 5μs (measured with TSC).
6. Zero heap allocations after initialisation (verified with a custom global new operator that counts calls).

5. Phase 2: Concurrent Pipeline — Lock-Free Multithreading

Weeks 9–16 • Goal: Decouple network I/O from processing using lock-free inter-thread communication, achieving deterministic latency.

5.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Thread Architecture	Split the system into dedicated threads: I/O Thread, Strategy Thread, and OMS Stub Thread.	Use std::jthread (C++20) for RAII-based thread management with cooperative cancellation via std::stop_token. Pin each thread to a specific CPU core using pthread_setaffinity_np(). Isolate cores from the OS scheduler using the isolcpus kernel parameter.	Thread pinning eliminates context-switch latency (typically 1–10μs per switch). Core isolation prevents the OS from scheduling other work on your critical cores. std::jthread's stop_token gives you clean shutdown without manual flag-checking.
SPSC Queue (Production)	Upgrade the warm-up SPSC queue to a production-grade implementation with batched operations.	Cache-line-padded head and tail indices to prevent false sharing. Store elements in a power-of-2-sized array for fast modulo (bitwise AND). Support try_push/try_pop (non-blocking) and push_batch/pop_batch for bulk operations. Add a monotonic sequence number per slot for ABA problem avoidance.	False sharing is the number one performance killer in naive multithreaded code. When two atomics share a cache line, every write by one thread invalidates the other's cache, causing a full cache coherency round-trip (50–100ns on modern CPUs). Padding to 64 bytes eliminates this.
Memory Ordering	Implement correct and minimal memory ordering on all atomic operations.	Producer uses std::memory_order_release on the tail write (ensures all prior writes to the data slot are visible before the tail advances). Consumer uses std::memory_order_acquire on the tail read (ensures it sees all writes from the producer). Relaxed ordering on local-only operations. Never use memory_order_seq_cst unless you can prove it's necessary.	Memory ordering is the hardest concept in lock-free programming. x86 has a strong memory model (stores are not reordered with respect to other stores), but ARM does not. Using release/acquire explicitly makes your code portable and documents your intent. seq_cst adds a full memory fence that costs 10–30ns on x86.
Object Pool (Production)	Provide allocation-free object creation for MarketDataEvent and OrderRequest.	A templated ObjectPool<T, N> that pre-allocates N objects in a contiguous array. The free list is intrusive: each free object contains a next	The system allocator (malloc) is a major source of latency jitter. Even modern allocators like jemalloc can take 100ns–100μs

Component	Objective	C++ Low-Level Focus	Why It Matters
		pointer (union with the object's first field). allocate() pops from the free list in O(1). deallocate() pushes back in O(1). Add debug-mode checks for double-free and use-after-free.	depending on contention and the size class. A fixed-size pool is O(1) with no contention (since each thread has its own pool).
Pipeline Plumbing	Wire the I/O thread to the strategy thread via the SPSC queue, passing MarketDataEvents.	I/O thread: recv() → parse() → pool.allocate() → fill event → queue.push(). Strategy thread: queue.pop() → update_book() → evaluate_strategy() → pool.deallocate(). Add backpressure detection: if the queue is > 80% full, log a warning (the strategy thread is falling behind).	This producer-consumer pattern is the backbone of every HFT system. The key insight is that the I/O thread never waits for the strategy thread, and the strategy thread never waits for the I/O thread. They communicate only through the lock-free queue. If one is slower, messages queue up — this is detectable and actionable.
Latency Measurement	Instrument the pipeline to measure and report per-message latency.	Stamp each event with a TSC timestamp at the moment the I/O thread completes recv(). When the strategy thread finishes processing, stamp again. The difference is the pipeline latency. Store measurements in a pre-allocated ring buffer of uint64_t. Every N seconds, compute p50/p90/p99/p99.9 from the buffer using a partial sort (std::nth_element).	Latency histograms are how you prove your system is fast. Average latency is meaningless — a system with 1μs average but 1ms p99.9 is unusable. Hedge fund interviewers will ask about tail latency. Being able to show measured percentiles from your own system is extremely impressive.

5.2 C++ Concepts Refresher for This Phase

std::atomic and Memory Ordering

std::atomic<T> provides atomic (indivisible) read-modify-write operations on T. The memory ordering parameter controls which other memory operations are guaranteed to be visible. memory_order_relaxed: only the atomic operation itself is atomic; no ordering guarantees for surrounding code. memory_order_acquire: all reads/writes after this load see the effects of the corresponding release store. memory_order_release: all reads/writes before this store are visible to the corresponding acquire load. Think of release as "publishing" and acquire as "subscribing."

False Sharing

Modern CPUs transfer memory in 64-byte cache lines. If thread A writes to variable X and thread B writes to variable Y, but X and Y are in the same 64-byte line, every write by A invalidates B's cached

> copy (and vice versa). This is called false sharing and can degrade performance by 10–50x. The fix is to pad structures so that data accessed by different threads lives on different cache lines. In C++, use alignas(64) or the C++17 std::hardware_destructive_interference_size constant.

5.3 Acceptance Criteria

1. Three threads running: I/O, Strategy, OMS Stub, each pinned to a dedicated core.
2. SPSC queue throughput > 100M messages/second in isolated benchmark.
3. End-to-end pipeline latency (recv to book update) < 2μs at p99 (measured).
4. Zero allocations during steady-state operation (verified with custom global operator new).
5. Clean shutdown via std::stop_token with no data loss.
6. Latency histogram reporting p50/p90/p99/p99.9 every 10 seconds to log.

6. Phase 3: Order Book Engine & Strategy Logic

Weeks 17–24 • Goal: Build a correct, high-performance order book and implement initial market-making and arbitrage detection strategies.

6.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
L2 Order Book Engine	Maintain a complete Level 2 order book (price-aggregated) with guaranteed correctness under high update rates.	Implement separate bid and ask sides using a sorted std::vector<PriceLevel> (flat, cache-friendly). For top-of-book access, maintain a denormalized best_bid/best_ask that is updated on every modification. Support apply_snapshot() and apply_delta() methods. Use a checksum/CRC to periodically verify against exchange-provided checksums.	A flat vector is 5–10x faster than std::map for < 100 levels because the entire side fits in a few cache lines. The exchange sends periodic checksums (Binance, Kraken, Bybit all do this) — verifying them is how you prove your book is correct.
Market-Making Strategy	Implement a basic market-making strategy that quotes two-sided markets around a fair-value estimate.	The strategy maintains a fair-value estimate (initially just the midpoint). It quotes a configurable number of levels on each side with spread and skew parameters. Inventory skew: as position grows long, widen the ask and tighten the bid to encourage flattening. All parameters stored in a cache-line-aligned config struct, updatable at runtime via atomic reads from a config-reload thread.	Market-making is the most common strategy at quant firms. Understanding how spread, skew, and inventory management interact is foundational knowledge. The runtime-updatable config pattern (atomic reads, no locks) is used everywhere in production systems.
Arbitrage Detector	Detect cross-exchange arbitrage opportunities by comparing best bid/ask across venues.	Compare best_bid[venue_A] against best_ask[venue_B] (and vice versa). Account for fees on both sides. Only signal when the expected profit exceeds a configurable threshold. This is a pure function: it takes two BookSnapshot structs and returns an std::optional<ArbitrageSignal>. No allocations, no side effects.	Cross-exchange arb is mathematically simple but operationally hard (latency matters, partial fills create risk). Starting with detection only (no execution) lets you measure how often opportunities appear and how long they last, which informs whether execution is viable.

Component	Objective	C++ Low-Level Focus	Why It Matters
Strategy Abstraction	Allow multiple strategies to be compiled into the same binary without virtual dispatch overhead.	Use CRTP (Curiously Recurring Template Pattern) to define a Strategy base. Each concrete strategy (MarketMaker, Arbitrageur) inherits from Strategy<MarketMaker>. The base class calls static dispatch via static_cast<Derived*>(this)->on_book_update(). This gives you polymorphism at compile time with zero runtime cost.	Virtual functions require a vtable lookup (pointer chase) on every call. In a hot loop processing millions of messages, this can add 5–15ns per call due to the indirect branch and potential cache miss. CRTP eliminates this entirely.
Signal / Order Request Generation	Create a minimal struct that carries a trade signal from the strategy to the OMS.	OrderRequest: {symbol_id, side, price, quantity, order_type (limit/market), time_in_force, strategy_id, timestamp}. Allocated from an ObjectPool. Pushed to the OMS queue via SPSC. Use an enum class for all categorical fields to prevent implicit conversions.	Keeping the signal struct minimal and POD ensures it fits in a cache line and can be passed through the lock-free queue without copies.

6.2 C++ Concepts Refresher for This Phase

CRTP (Curiously Recurring Template Pattern)

A pattern where a class inherits from a template instantiated with itself: class MarketMaker : public Strategy<MarketMaker>. The base class Strategy<T> can call methods on the derived class via static_cast<T*>(this)->method(). This resolves at compile time (no vtable, no indirect branch), giving you the extensibility of inheritance with the performance of static dispatch. The downside is that all strategy types must be known at compile time — you can't load a new strategy from a shared library at runtime. For a personal project, this is a fine trade-off.

constexpr and Compile-Time Computation

The constexpr keyword tells the compiler that a function or variable can be evaluated at compile time. For example, constexpr int64_t PRICE_SCALE = power(10, 8) computes the scale factor during compilation, not at runtime. In strategy code, fee calculations, threshold comparisons, and other constants can be constexpr, meaning the compiler embeds the result directly as an immediate value in the instruction stream.

6.3 Acceptance Criteria

1. L2 order book passes checksum verification against exchange reference (Binance checksum or Bybit CRC).
2. Market-making strategy generates correct quote updates at > 10,000 updates/second.
3. Arbitrage detector correctly identifies known historical opportunities from replayed data.

4. Zero virtual function calls on the hot path (verified with perf's branch-mispredict counter).
5. Book update latency < 200ns for a single level update (benchmarked).

7. Phase 4: Execution Engine & Order Management System

Weeks 25–34 • Goal: Build the component that sends real orders to exchanges, tracks their lifecycle, and manages positions. This is where the system goes from read-only to read-write.

> **This Is the Hard Phase**
>
> Phases 1–3 are conceptually clean: data flows one way, correctness is verifiable against the exchange, and bugs are observable. Phase 4 introduces bidirectional communication with an exchange that can reject, partially fill, or silently drop your orders. Bugs here can cost real money. Start on testnet and stay on testnet until Phase 5 (Risk Management) is complete.

7.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Order Lifecycle FSM	Model the complete lifecycle of an order as a finite state machine.	States: Created → PendingNew → Acknowledged → PartiallyFilled → Filled | Cancelled | Rejected. Use an enum class for states and a transition table (constexpr std::array of valid transitions). Every state transition is logged with timestamps. Invalid transitions trigger an immediate error and position reconciliation.	Order state machines are the single most interview-tested component at trading firms. The key insight is that the exchange is the source of truth for order state, not your system. Your local state machine must handle every possible exchange response, including ones you didn't expect (e.g., a fill arriving after you thought the order was cancelled).
Exchange Gateway	Abstract the exchange-specific REST/WebSocket API behind a clean interface.	A templated ExchangeGateway<Venue> class (CRTP again) that implements: submit_order(), cancel_order(), cancel_replace(), and query_order(). Each venue specialisation handles the specific API format (Binance POST /api/v3/order, etc.). HTTP calls use a pre-created connection pool (persistent connections, no TCP handshake per order). Parse fill reports from the WebSocket user data stream.	Connection pooling eliminates TCP handshake latency (~1ms) from the order submission path. Abstracting behind a template lets you add new venues (Bybit, OKX, Polymarket) without changing strategy code.
Position Tracker	Track real-time position, average entry price, and unrealised P&L per instrument.	A Position struct: {int64_t quantity (signed: positive=long, negative=short), int64_t avg_entry_price, int64_t	Signed quantity with FIFO cost basis is the standard position model. By keeping the position on a single thread (the OMS thread),

Component	Objective	C++ Low-Level Focus	Why It Matters
		realised_pnl, int64_t unrealised_pnl}. Updated on every fill. unrealised_pnl is recalculated on every book update using the current mid-price. Thread-safe via the warm-path thread owning the position exclusively (no sharing, no atomics needed).	you avoid all synchronization complexity.
Order ID Management	Generate locally unique order IDs and map them to exchange-assigned IDs.	Use a per-session monotonic counter for local order IDs (uint64_t, never reused within a session). Maintain a pre-allocated flat_hash_map<local_id, exchange_id> for cross-referencing. Also maintain an active_orders array (fixed capacity, linear scan — you'll rarely have > 20 active orders).	Exchange IDs are strings on most venues, which means allocation if you store them naively. Use a fixed-size char array (e.g., char[32]) inside the order struct to avoid this.
Fill Handler	Process fill reports from the exchange, update positions, and reconcile with expected state.	On fill: validate order ID exists in active set, transition FSM to PartiallyFilled or Filled, update position, update realised P&L, log the fill with full details. On unexpected fill (unknown order ID): trigger an alert, pause trading, and force a position reconciliation via REST.	Unexpected fills happen more often than you'd think (exchange retransmissions, network duplicates, race conditions during cancellation). A system that panics on unexpected fills is fragile; a system that reconciles and continues is production-grade.

7.2 Testnet Protocol

Do not skip this. The testnet phase is where you find the bugs that would cost money in production.

1. Run the full system on Binance Testnet or Bybit Testnet for at least 2 weeks before any mainnet activity.
2. Simulate adverse conditions: manually cancel orders on the exchange web UI while your system expects them active. Send duplicate orders. Disconnect the WebSocket mid-order.
3. Verify that every state transition is handled: submit → ack, submit → reject, partial fill → cancel, fill arriving after cancel sent.
4. Log every order lifecycle event with microsecond timestamps. Review logs daily for anomalies.

7.3 Acceptance Criteria

1. Order lifecycle FSM handles all valid transitions and rejects invalid ones.

2. Round-trip order latency (submit to ack) measured and logged, typically 5–50ms for REST.
3. Position tracker agrees with exchange-reported position after 24 hours of continuous testnet trading.
4. Unexpected fill handling tested and verified (system pauses and reconciles, does not crash).
5. Zero memory leaks (verified with AddressSanitizer over a 24-hour run).

8. Phase 5: Risk Management & Production Hardening

Weeks 35–42 • Goal: Make the system safe to run with real money by adding comprehensive risk controls, async logging, monitoring, and crash recovery.

8.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Pre-Trade Risk Gateway	Validate every order against risk limits before it reaches the exchange.	A synchronous check on the warm path (before exchange submission): max order size, max position per instrument, max total exposure, max orders per second (rate limit), price sanity check (reject orders > X% from mid-price). All limits stored in a RiskConfig struct with atomic reads for runtime updates. A rejected order logs the reason and notifies the strategy.	Pre-trade risk checks are legally required at most regulated venues and universally expected in quant interviews. The key design point is that these checks must be fast enough to not add meaningful latency to the order path, but thorough enough to prevent catastrophic errors.
Kill Switch	Provide an immediate, unconditional mechanism to cancel all orders and halt trading.	A global std::atomic<bool> kill_switch flag. When set: (1) all pending orders are cancelled via bulk cancel API, (2) no new orders are accepted, (3) all strategies are notified to wind down. Triggered by: max daily loss exceeded, anomaly detection (e.g., position diverging from expected), manual signal (SIGUSR1). Also implement a "soft" kill that just stops new orders but lets existing ones fill.	The kill switch must work even if the strategy code is buggy or hung. That's why it's a global atomic checked independently by the OMS before every exchange API call, not a callback into strategy code.
Async Logger (MPSC)	Replace the Phase 1 logging with a high-throughput, zero-contention async logger.	All threads push log messages to a Multi-Producer Single-Consumer (MPSC) lock-free queue. A dedicated logger thread drains the queue and writes to disk. Log messages are pre-formatted by the producer (using fmt::format_to into a stack buffer) and moved into the queue as fixed-size blocks. Use a binary log format (not text) for the hot-path logs —	Text formatting is expensive (100ns–1μs). Binary logging pushes the formatting cost to read-time instead of write-time. The MPSC queue allows multiple producer threads without contention (each producer CAS's the tail pointer).

Component	Objective	C++ Low-Level Focus	Why It Matters
		a separate tool decodes them for human reading.	
Latency Histogram	Track and report latency percentiles for every stage of the pipeline.	A custom histogram with logarithmically-spaced buckets (e.g., 1ns, 2ns, 4ns, ..., up to 1s). On each measurement, increment the appropriate bucket counter (a simple array index operation — O(1), no allocation). Every 10 seconds, compute p50/p90/p99/p99.9 from the cumulative distribution. Export to a Prometheus endpoint.	This is your system's telemetry. Without it, you're flying blind. The logarithmic bucketing gives good resolution at both low and high latencies without consuming excessive memory.
State Snapshots	Save the complete system state (positions, active orders, configuration) to disk for fast recovery.	Use memory-mapped files (mmap) for the position and order state. On shutdown (or periodically), msync the mapped region to disk. On restart, re-map the file and resume from the last snapshot. Add a checksum (CRC32) to detect corruption.	mmap avoids serialization/deserialization entirely — the in-memory struct IS the on-disk format. Recovery is near-instantaneous (just map the file). This is how real trading systems handle restarts.
Anomaly Detection	Automatically detect and respond to conditions that suggest the system is misbehaving.	Monitor: (1) position divergence (local position vs exchange-reported), (2) order rate exceeding expectations, (3) strategy P&L exceeding daily bounds, (4) feed latency exceeding threshold. Each anomaly has a configured response: log-only, soft-kill, or hard-kill.	Autonomous anomaly detection is what separates a demo from a production system. It's also a major interview topic: "How would you detect and respond to a strategy that has gone haywire?"

8.2 Mainnet Graduation Criteria

Do Not Go Live Until All of These Are Met

1. All risk checks pass a comprehensive test suite (including edge cases like max-int positions, zero-price orders). 2. Kill switch has been tested by triggering it mid-trade on testnet and verifying clean shutdown. 3. System has run 7+ consecutive days on testnet with no unexpected crashes, no position divergence, and no unhandled errors. 4. State snapshot/recovery has been tested by killing the process with SIGKILL and verifying correct recovery. 5. Daily loss limit has been tested by simulating a loss exceeding the limit and verifying trading halts.

9. Phase 6: Multi-Venue Expansion & Polymarket Integration

Weeks 43–52 • Goal: Connect to multiple exchanges and prediction markets, normalize data across venues, and enable cross-venue strategies.

9.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Venue Abstraction Layer	Provide a uniform interface across all connected venues so strategies are venue-agnostic.	A VenueTraits<V> template that defines per-venue types: message format, order ID type, fee structure, rate limits. Each venue specialisation (Binance, Bybit, Polymarket) implements the traits. The strategy layer only interacts with normalised types — it never knows which venue it's trading on.	This is the software engineering phase. Clean abstractions here pay dividends when adding new venues. A well-designed venue layer means adding OKX or dYdX is a 2-day task, not a 2-week one.
Multi-Feed Aggregator	Consume and synchronize market data from multiple venues simultaneously.	Each venue runs its own I/O thread (pinned to separate cores). All feeds publish to a shared MPSC merge queue. The strategy thread consumes from this merged stream. Timestamps are normalized to a common clock. A VenueBook struct holds one order book per venue, and a CompositeBook provides best-bid/best-ask across all venues.	Real cross-exchange arbitrage requires comparing books from different exchanges with sub-millisecond accuracy. Clock synchronization across venues is non-trivial — each exchange has different latency characteristics. NTP is your friend here.
Polymarket Integration	Connect to Polymarket's CLOB API and trade binary outcome contracts.	Polymarket uses a REST + WebSocket API similar to crypto exchanges, but assets are binary contracts (YES/NO shares priced 0.00–1.00). Implement the venue specialisation: parse the order book, submit limit orders, handle fills. Shares settle on-chain (Polygon), so you'll need a wallet integration for settlement (but trading is off-chain via the CLOB).	Polymarket is the killer application for the AI layer (Phase 7). The competition is unsophisticated compared to crypto markets. Wide spreads and retail participants mean that even a basic market-maker can capture meaningful profit. More importantly, the AI's ability to better estimate event probabilities becomes direct alpha.
Cross-Venue Execution	Execute arbitrage legs across two venues	When the arbitrage detector signals (bid[A] > ask[B] + fees): simultaneously submit	"Leg risk" is the defining challenge of cross-exchange arb. One

Component	Objective	C++ Low-Level Focus	Why It Matters
	atomically (or as close to atomically as possible).	a sell to A and a buy to B. Use non-blocking order submission (fire both orders, then await acknowledgements). Handle the partial-fill case: if only one leg fills, you have unhedged exposure — immediately attempt to flatten via the other venue or by unwinding the filled leg.	leg fills and the other doesn't, leaving you with directional exposure you didn't want. How you handle this is a major interview topic and a real source of losses.
Fee Normalisation	Accurately model fee structures across venues to avoid arbing yourself into a loss.	Each venue has a different fee model: maker/taker percentages, tiered by volume, sometimes with token-based discounts (BNB for Binance). Implement a FeeModel<Venue> that computes expected fee for a given order. All arb calculations subtract worst-case fees from both legs before signalling.	Many hobby arb systems ignore fees and show phantom opportunities that are actually net-negative. Accurate fee modeling is unglamorous but essential.

9.2 Polymarket-Specific Notes

Polymarket is a prediction market built on the Polygon blockchain. Here are the key technical details for integration:

API: CLOB (Central Limit Order Book) accessible via REST and WebSocket. The API is similar to a crypto exchange — order books, order submission, fills, cancellation all work the same way. Documentation at docs.polymarket.com.

Assets: Binary outcome contracts. Each market (e.g., "Will X happen by Y date?") has YES and NO tokens. Prices range from $0.01 to $0.99. If the event occurs, YES tokens settle at $1.00 and NO tokens at $0.00.

Settlement: On-chain on Polygon. You need a funded wallet (USDC) for collateral. Trading itself is off-chain via signed orders.

Opportunities: Spreads are often 2–5% on less liquid markets. Informational edge (better probability estimates) can generate significant returns on markets where the crowd is wrong.

10. Phase 7: AI/ML Signal Layer & Agent Orchestration

Weeks 53–64 • Goal: Add an intelligence layer that generates fair-value estimates and strategic decisions, bridging your Python ML skills with the C++ execution engine.

10.1 Architecture

The AI layer runs in Python (leveraging your current role's skillset) and communicates with the C++ engine via shared memory. This is a deliberate design choice: ML model development is vastly more productive in Python, and the communication overhead (shared memory read is < 100ns) is negligible compared to the signal's time horizon (seconds to minutes).

10.2 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Shared Memory IPC	Establish a high-speed communication channel between Python and C++.	Use POSIX shared memory (shm_open + mmap) to create a shared region. Define a fixed-layout SignalMessage struct (fair_value, confidence, timestamp, model_id) in both C++ and Python (use ctypes or struct module). The Python side writes; the C++ side reads. Use an atomic sequence counter for consistency (writer increments after write, reader checks before and after read to detect torn writes).	Shared memory is the fastest possible IPC mechanism — it's literally the same physical memory accessed by both processes. The sequence counter pattern (called a seqlock) provides consistency without locks. This is used in the Linux kernel and in production trading systems.
Order Flow Signal	Train an ML model that predicts short-term price direction from order book features.	Features: bid-ask imbalance, order flow imbalance (volume of market buys vs sells over trailing window), spread changes, book pressure (total bid size vs ask size). Model: start with a gradient-boosted tree (XGBoost or LightGBM) trained on historical tick data that your system has been collecting since Phase 1. Export as ONNX and load in C++ for inference, OR keep in Python and publish signals via shared memory.	Order flow prediction is one of the simplest alpha signals and a great starting point. The key is that your system has been collecting the training data since Phase 1 — you're not relying on third-party data.
Prediction Market Model	Estimate true probabilities for Polymarket events using external data sources.	For each market category, build a specialised model: (1) Sports: use historical data and live game state (APIs:	This is where the real money is. If your model says an event has a 65% chance of occurring but Polymarket

Component	Objective	C++ Low-Level Focus	Why It Matters
		ESPN, The Odds API) to estimate win probabilities. (2) Politics: aggregate polling data, prediction market prices from other venues (Kalshi, Betfair), and base rates. (3) Crypto events: use on-chain data and sentiment. Publish probability estimates to shared memory. The C++ engine quotes around these fair values.	prices it at 55%, you buy YES shares and capture the expected value. Unlike crypto market-making where your edge is speed, here your edge is information — which is durable and scalable.
Agent Orchestrator	Build an LLM-based agent that makes high-level strategic decisions.	A Python agent (using Claude API or local model) that: (1) monitors overall system health and P&L, (2) decides which markets to enter/exit based on current conditions, (3) adjusts risk parameters based on market regime (low-vol vs high-vol), (4) processes unstructured information (news, social media) to assess event probabilities. The agent publishes configuration updates (which pairs to trade, risk limits) to the C++ engine via shared memory.	This is the cutting-edge layer that almost nobody has. An LLM agent that manages a portfolio of trading strategies, adjusting allocation based on both quantitative signals and qualitative information processing. This is the kind of system architecture that hedge funds are actively researching.

10.3 C++ / Python Interface Detail

The shared memory layout is versioned and strictly typed. Here is the conceptual struct:

SignalBuffer (shared memory, 4KB page-aligned): version (uint32_t), sequence (atomic<uint64_t>), num_signals (uint32_t), signals[MAX_SIGNALS] where each signal is: symbol_id (uint16_t), fair_value (int64_t fixed-point), confidence (uint16_t 0–10000 representing 0–100.00%), model_id (uint16_t), timestamp (uint64_t nanoseconds).

The C++ reader uses a seqlock pattern: read sequence, read data, read sequence again. If sequences don't match, the writer was mid-update — retry. This is wait-free for the writer and lock-free for the reader.

11. Phase 8: Replay, Backtesting & Performance Lab

Weeks 65–72 • Goal: Build the infrastructure to replay historical data through the live system, enabling backtesting and performance regression testing.

11.1 Component Breakdown

Component	Objective	C++ Low-Level Focus	Why It Matters
Binary Data Logger	Record all market data and order events in a compact binary format for replay.	Use SBE (Simple Binary Encoding) or FlatBuffers for the log format. Each record has a header (timestamp, event_type, length) followed by the event payload. Write to memory-mapped files in large sequential blocks (4MB) for maximum disk throughput. Rotate files daily.	Text logs (JSON, CSV) are 10–100x larger and 10–100x slower to parse than binary formats. SBE is the industry standard in financial systems for exactly this reason. FlatBuffers is a good alternative with better tooling.
Replay Harness	Feed historical data through the exact same code path as live trading.	Replace the WebSocket I/O thread with a ReplayFeed that reads from binary log files and pushes events to the same SPSC queue at a configurable speed (1x real-time, max-speed, or stepped). The strategy and OMS threads don't know (and don't care) whether data is live or replayed. This "one binary, two modes" design is a hallmark of professional systems.	If your backtest code path diverges from your live code path, your backtest results are meaningless. Replaying through the same pipeline guarantees consistency. This also lets you regression-test performance: replay the same data after a code change and verify latency hasn't degraded.
Performance Lab	Systematically profile and optimise the system's critical path.	Use perf stat to measure IPC (instructions per cycle), cache miss rate, and branch misprediction rate. Use perf record + perf report to find hot functions. Use cachegrind for detailed cache simulation. Target: > 2.0 IPC on the hot path, < 1% L1d cache miss rate, < 0.5% branch misprediction rate. Document every optimisation with before/after measurements.	This is the phase that makes the difference in interviews. Being able to say "I reduced p99 latency from 3.2μs to 1.8μs by restructuring the book update to be branch-free and reducing the working set to fit in L1" is exactly the kind of story that gets offers at top firms.
Regression Testing	Ensure code changes don't degrade performance.	Add a CI benchmark suite that replays 1 hour of historical data and measures p50/p90/p99/p99.9 latency. If any percentile degrades by > 10% vs the baseline, the CI pipeline fails. Use Google	Performance regression testing is surprisingly rare in hobby projects and very common in production systems. Having it signals maturity.

Component	Objective	C++ Low-Level Focus	Why It Matters
		Benchmark for microbenchmarks of individual components (queue, allocator, book update). Store benchmark results in version control for trend tracking.	

12. C++ Refresher Reference — Key Concepts

A concise reference for the C++ features used most heavily throughout TACHYS. Designed for someone returning to C++ after a year in Python.

Move Semantics

When you "move" an object (e.g., std::move(x)), you transfer ownership of its internal resources (heap memory, file handles) to a new owner. The old object is left in a valid but unspecified state. This avoids expensive deep copies. In TACHYS, move semantics are used when passing MarketDataEvents through queues — we move the event in, not copy it. Key syntax: MyClass(MyClass&& other) noexcept is the move constructor. Mark it noexcept or containers like std::vector won't use it.

RAII

Resource Acquisition Is Initialization. Every resource (socket, file, memory, lock) is tied to an object's lifetime. The constructor acquires the resource; the destructor releases it. You never manually close/free/release anything. In TACHYS, sockets, memory-mapped files, and thread handles are all RAII-managed.

std::span<T>

A non-owning view of a contiguous sequence of T (like std::string_view but for any type). Replaces the C-style (pointer, length) pair. Use it for function parameters when you want to accept arrays, vectors, or raw buffers without caring about the container. Zero overhead.

std::optional<T>

A type that either contains a T or is empty (std::nullopt). Replaces the pattern of using a sentinel value (-1, nullptr) to indicate "no result." In TACHYS, functions like detect_arbitrage() return std::optional<ArbitrageSignal> — either an opportunity exists or it doesn't.

std::expected<T, E>

C++23. Like std::optional but the "empty" case carries an error value E. Replaces exceptions for error handling on the hot path. parse_message() returns std::expected<MarketDataEvent, ParseError> — either a valid event or a typed error. No exceptions, no heap allocation, no stack unwinding.

enum class

A scoped enumeration that doesn't implicitly convert to int. enum class Side : uint8_t { Buy, Sell }; takes exactly 1 byte, can't accidentally be compared to an int, and is type-safe. Use everywhere you'd use an enum in C.

constexpr if

Conditional compilation in templates: if constexpr (std::is_same_v<Venue, Binance>) { ... }. The unused branch is discarded at compile time, not evaluated. Used in venue-templated code to specialise behaviour without runtime cost.

Structured Bindings

auto [price, quantity] = get_best_bid(); — unpacks a struct or tuple into named variables. Python-like syntax, zero overhead.

std::atomic<T>

A type whose operations (load, store, exchange, compare_exchange) are indivisible at the hardware level. Essential for lock-free programming. Always specify the memory ordering explicitly (acquire, release, relaxed) rather than relying on the default (seq_cst).

Concepts (C++20)

Named constraints on template parameters. template<typename T> concept OrderBookSide = requires(T t) { t.best_price(); t.update(level); }; — gives clear error messages when a type doesn't meet requirements, instead of 500 lines of template instantiation errors.

13. AWS Deployment Architecture

The system is designed to run on AWS for proximity to exchange matching engines and for reliable uptime. Here is the recommended setup.

Primary Instance	c6i.2xlarge (8 vCPU, 16GB RAM) in ap-northeast-1 (Tokyo) for Binance proximity, or us-east-1 for Polymarket/US exchanges. Use a Dedicated Host or metal instance for core isolation (isolcpus).
Operating System	Ubuntu 22.04 LTS with the PREEMPT_RT kernel patch for deterministic scheduling. Disable transparent huge pages (THP), disable CPU frequency scaling (set to performance governor), disable hyper-threading on the cores used for the hot path.
Networking	Enable Enhanced Networking (ENA) for < 25μs network latency within AZ. Use placement groups for instances that need to communicate. Consider AWS Global Accelerator for cross-region communication if trading on exchanges in different geographies.
Storage	io2 Block Express EBS for binary logs (up to 64,000 IOPS). Use instance store (NVMe) for temporary data and memory-mapped state files (fastest possible I/O).
Monitoring	CloudWatch for infrastructure metrics. Prometheus + Grafana running on a separate t3.small for application metrics (latency histograms, P&L, position). Export from the C++ monitoring thread via a simple HTTP endpoint.
Cost Estimate	c6i.2xlarge on-demand: ~$250/month. With 1-year reserved: ~$155/month. Add ~$50/month for storage and monitoring. Total: ~$200–$300/month, easily covered by modest trading revenue.

14. Revenue Model & Realistic Expectations

This section provides honest, conservative estimates. Crypto and prediction market trading involves real risk of loss. Never trade with money you cannot afford to lose.

14.1 Revenue Streams

Crypto Market-Making	Focus on mid-cap tokens (rank 50–200) on Tier 1–2 exchanges where spreads are 10–30 bps and competition is thin. Realistically capture 3–8 bps per round-trip. At $100K–$500K daily notional volume, this generates $30–$400/day gross. After fees and inventory risk: $10–$200/day net. Monthly range: $300–$6,000.
Cross-Exchange Arbitrage	Opportunities are spiky: frequent during volatile periods, rare during calm markets. Average 5–20 actionable signals/day with $5–$50 profit per arb (after fees). Monthly range: $100–$1,000.
Polymarket Market-Making	Spreads are wide (2–5%) on less liquid markets. Lower notional but higher margins. With AI-derived fair values, directional edge adds to market-making revenue. Monthly range: $200–$2,000.
AI-Driven Directional	When your model disagrees with the market and is right more often than not, directional positions capture the mispricing. This is the highest-variance, highest-potential stream. Monthly range: -$500 to +$5,000 (genuinely variable, can lose money).

14.2 Realistic Timeline to Revenue

Months 1–6: Zero revenue. You are building infrastructure. Months 6–9: Testnet only. Still zero revenue, but you are validating the system. Months 9–12: First cautious mainnet deployment with minimal capital ($1K–$5K). Revenue: $0–$500/month. Months 12–18: Scale up capital and add Polymarket + AI layer. Revenue: $500–$5,000/month with significant variance. AWS costs are covered within the first month of mainnet trading in all but the worst scenarios.

14.3 Risk Acknowledgement

> **This is not a get-rich-quick scheme**
>
> You can and will lose money during the learning phase. Exchange API bugs, network outages, flash crashes, regulatory changes, and plain old software bugs can all cause losses. The risk management layer (Phase 5) exists to bound these losses. Start with capital you are prepared to lose entirely. The primary value of this project is education and career advancement; revenue is a secondary benefit.

15. Appendices

15.1 Recommended Reading

C++ Concurrency	C++ Concurrency in Action (2nd ed.) by Anthony Williams. The definitive reference for std::atomic, memory ordering, and lock-free data structures.
Low-Latency Systems	Trading and Exchanges: Market Microstructure for Practitioners by Larry Harris. Not C++ specific, but essential for understanding order books and market-making.
Systems Programming	The Linux Programming Interface by Michael Kerrisk. Reference for sockets, mmap, signals, and all the Linux APIs you will use.
Performance	What Every Programmer Should Know About Memory by Ulrich Drepper (free PDF). Essential for understanding cache hierarchy and memory layout optimisation.
Modern C++	A Tour of C++ (3rd ed.) by Bjarne Stroustrup. The fastest way to catch up on C++17/20/23 features.

15.2 Exchange API References

Binance	WebSocket Streams: binance-docs.github.io/apidocs/spot/en. Testnet: testnet.binance.vision. L2 depth stream with 100ms updates. Checksum verification available.
Bybit	docs.bybit.com. Good WebSocket API with delta updates and checksums. Lower latency than Binance for some pairs.
Polymarket	docs.polymarket.com. CLOB API documentation. REST for order management, WebSocket for book updates. Polygon blockchain for settlement.

15.3 Interview Preparation Mapping

Every phase of TACHYS directly maps to common interview topics at quantitative trading firms. Use this mapping to prepare talking points:

"Describe your experience with lock-free data structures"	Phase 2: SPSC queue, Phase 5: MPSC logger. Discuss memory ordering, false sharing, ABA problem.
"How do you handle memory allocation in latency-sensitive code?"	Phase 2: Object pool. Phase 1: Pre-allocated buffers. Discuss arena allocation, pool allocation, and why malloc is non-deterministic.
"Walk me through an order book implementation"	Phase 3: Flat vector book with checksum verification. Discuss why std::map is wrong, cache line analysis, and the correctness guarantees.
"How do you measure and reduce tail latency?"	Phase 5: Latency histogram. Phase 8: Performance lab. Discuss p99.9 measurement, perf profiling, cache miss analysis, branch-free code.

"Tell me about a bug in a concurrent system"	Phase 4/5: Real war stories from testnet and mainnet. Discuss race conditions, unexpected fills, state machine edge cases.
"How do you design a risk management system?"	Phase 5: Pre-trade risk, kill switch, anomaly detection. Discuss the philosophy: defence in depth, fail-safe defaults, independent risk thread.